

Lisp, Lore, and Logic: An Algebraic View of Lisp Programming, Foundations, and Applications

W. Richard Stark

Preface

Acknowledgments

Chapter 0 Introduction

Chapter 1 A First Look at LISP

- 1.1 The origins of LISP
- 1.2 A glimpse into the structure of LISP
 - 1.2.1 Data
 - 1.2.2 Functions
 - 1.2.3. Expressions in the language
 - 1.2.4 QUOTE
 - 1.2.5 Logical Terms
 - 1.2.6 Branching in computations
 - 1.2.7 Lambda functions
 - 1.2.8 Assignments
- 1.3 Additional reading

Chapter 2 Elementary Programming

- 2.1 Programming straightforward computations
- 2.2 Recursive and iterative computations
 - 2.2.1 A general view of recursion
 - 2.2.2 Programming recursive computations
 - 2.2.3 Rules for recursive programming
 - 2.2.4 Iteration
- 2.3 External files
- 2.4 Searching
 - 2.4.1 Is searching a sign of ignorance?
 - 2.4.2 Kinds of search
- 2.5 Extra variables and functions, for efficiency

- 2.6 Other list functions
 - 2.6.1 MEMBER and more
 - 2.6.2 Functions for association lists
 - 2.6.3 Example: translation between Inuit and English
- 2.7 The fixed-point style of computation
- 2.8 Input and output
- 2.9 The procedural and functional styles of computation
- 2.10 Additional reading

Chapter 3 Deeper into Essential Structure

- 3.1 LISP's data
 - 3.1.1 S-expressions
 - 3.1.2 Representations of data
 - 3.1.3 Atoms
 - 3.1.4 Data types
- 3.2 Another look at familiar functions
 - 3.2.1 CAR, CDR, and CONS: an alternate semantics
 - 3.2.2 EQ and EQUAL
 - 3.2.3 Options for LAMBDA functions
 - 3.2.4 SETF
- 3.3 LISP syntax in detail
 - 3.3.1 Terms
 - 3.3.2 Variables and forms
 - 3.3.3 Syntax puzzles
- 3.4 Self-processing
- 3.5 Bindings, scopes, and environments
- 3.6 Additional reading

Chapter 4 Computational Philosophy

- 4.1 Models of computation
 - 4.1.1 Mathematical formalisms for computation
 - 4.1.2 Church's thesis and computational completeness
- 4.2 Pure LISP
- 4.3 Types of recursion

- 4.3.1 Fat recursion and tail recursion
- 4.3.2 Compound recursion
- 4.3.3 Monotonic and nonmonotonic recursion
- 4.4 The limits of LISP: an unsolvable problem
- 4.5 The folklore of fixed-point computation
- 4.6 Additional reading

Chapter 5 LISP Functions for Powerful Programming

- 5.1 Debugging tools
 - 5.1.1 The listener, access to local bindings, and BACKTRACE
 - 5.1.2 STEP and TRACE
 - 5.1.3 Other tools for debugging
 - 5.1.4 Application: debugging a Lisp text (optional)
- 5.2 Applicative operators
 - 5.2.1 APPLY, FUNCALL, MAPCAR, and MAPLIST
 - 5.2.2 Application: symbolic differentiation
- 5.3 Macros
 - 5.3.1 The basic idea
 - 5.3.2 Applications: INFIX and EVAL&TIME
- 5.4 Structures, vectors, and arrays
 - 5.4.1 Vectors and arrays
 - 5.4.2 Structures
 - 5.4.3 Application: working with organic molecules
- 5.5 Function closures
 - 5.5.1 Creating and using closures
 - 5.5.2 Application: COMPOSE and Currying CONS
- 5.6 COERCion
- 5.7 Surgical operations

Chapter 6 Interpreters: from Algebra to LISP

- 6.1 Algebras and interpreting: an abstract view
 - 6.1.1 Algebras of objects and algebras of terms
 - 6.1.2 Interpretation and computation
- 6.2 LISP interpreters

- 6.2.1 A sketch of the usual LISP interpreter
- 6.2.2 A call-by-name interpreter for LISP
- 6.2.3 Put the interpreter inside, keep the user out
- 6.3 Compiled LISP

Chapter 7 Mathematical Foundations of LISP

- 7.1 The lambda calculus defined
- 7.2 Straightforward computation in the lambda calculus
- 7.3 Fixed-points for object functions
- 7.4 Recursive functions in the lambda calculus
- 7.5 Pure LISP in the lambda calculus
- 7.6 Additional reading

Chapter 8 Automatic Reasoning, Algebraic Intelligence

- 8.1 Logics
 - 8.1.1 Propositional logic
 - 8.1.2 Fundamental theorems for the propositional calculus
 - 8.1.3 Other approaches to automatic reasoning in PC
- 8.2 Predicate logic and unification
- 8.3 PROLOG
- 8.4 Speculation on algebraic intelligence
- 8.5 Additional reading

Chapter 9 Bibliography

Chapter 10 Answers to Selected Exercises

Chapter 11 Index